

# BDT training and application

07/03/2026

## Samples Used

| Sample       | Description                       | Source                  |
|--------------|-----------------------------------|-------------------------|
| <b>Data</b>  | 2018 UL SingleMuon                | Real collision data     |
| <b>DY</b>    | Drell-Yan $Z/\gamma^*$ to tau-tau | MC simulation           |
| <b>ttbar</b> | tt to tau-tau + X                 | MC simulation           |
| <b>QCD</b>   | Multi-jet background              | Data-driven (same-sign) |

# Processing Pipeline

NanoAOD --> Phase0 --> Phase1 --> Merge --> Proton Mixing --> Plots

## Scripts

| Stage  | Data                | DY          | ttbar           | QCD          |
|--------|---------------------|-------------|-----------------|--------------|
| Phase0 | fase0_data_mutau.py | fase0_dy.py | fase0.py        | -            |
| Phase1 | fase1_data.py       | fase1_dy.py | fase1_ttjets.py | fase1_qcd.py |

## Phase1: Selection cuts

| Variable        | Cut                 | Description                |
|-----------------|---------------------|----------------------------|
| muon_id         | $> 3$               | Medium MVA ID              |
| tau_id1 (VSjet) | $> 63$              | VeryTight DeepTau vs jets  |
| tau_id2 (VSe)   | $> 7$               | Tight DeepTau vs electrons |
| tau_id3 (VSmu)  | $> 1$               | Loose DeepTau vs muons     |
| muon_pt         | $> 35 \text{ GeV}$  | muon transverse momentum   |
| tau_pt          | $> 100 \text{ GeV}$ | tau transverse momentum    |
| charge product  | $< 0$               | opposite sign              |
| Delta R         | $> 0.4$             | Angular separation         |

**Note:** QCD uses **same-sign** (SS) selection

## Where event weights are applied (cont)

### ttbar

Phase1:  $\text{event\_weight} = 0.15 \times \text{muon\_SFs}$   $\leftarrow$  0.15 included here

Proton Mixing:  $\text{weight} = \text{event\_weight} \times 0.13$

Plotting:  $\text{final\_weight} = \text{weight} \times 1.0$

### QCD (data-driven)

Phase1:  $\text{weight}=1.0 \rightarrow$  Plotting:  $\text{final\_weight}=1.0$

# Cross-Section normalization factors

## Formula used

$\text{weight} = N_{\text{MC}} / N_{\text{expected}}$   
where  $N_{\text{expected}} = \text{Luminosity} \times \text{Cross-section}$

## Values used

| Sample       | Factor | Included Where?                 |
|--------------|--------|---------------------------------|
| <b>DY</b>    | 1.81   | Applied in plotting             |
| <b>ttbar</b> | 0.15   | Already in Phase 1 event_weight |

This does not change the outcome but should be fixed for consistency

# Single Lepton Plots: Cuts Applied

## Proton requirements

| Sample         | Cut                                                                                         |
|----------------|---------------------------------------------------------------------------------------------|
| Data/QCD       | <code>Sum\$(proton_multi_arm==0)&gt;0 &amp;&amp;<br/>Sum\$(proton_multi_arm==1)&gt;0</code> |
| MC (DY, ttbar) | <code>xi_arm1_1 &gt;= 0 &amp;&amp; xi_arm2_1 &gt;= 0</code>                                 |
| Signal         | <code>xi_arm1_1 &gt;= 0 &amp;&amp; xi_arm2_1 &gt;= 0</code>                                 |

# Weights applied

## Weights applied

| Sample | Weight                                                                                             |
|--------|----------------------------------------------------------------------------------------------------|
| Data   | None                                                                                               |
| QCD    | None                                                                                               |
| DY     | Proton acceptance (0.245) - xsec normalization already included                                    |
| ttbar  | Proton acceptance (0.245) $\times$ Scale(0.15)                                                     |
| Signal | weight branch (lumi $\times$ xsec $\times$ SFs $\times$ rad. damage) $\times$ 5000 (visualization) |



# Weight fix: old vs new approach

## Old weights (MuTauPlots\_weight\_comparison.C)

- MC filled with **flat** weight = 0.245 (proton acceptance, no weight branch)
- Then Scale(1.81) for DY, Scale(0.15) for ttbar
- No generator\_weight, no muon scale factors

## New weights (corrected)

- MC filled using **weight branch** = generator\_weight x muon\_SF's x 0.245
- Then Scale(1.004e-4) for DY (= L x sigma / Sum\_w), Scale(1.0) for ttbar

|                     | Old                 | New                                       |
|---------------------|---------------------|-------------------------------------------|
| <b>DY weight</b>    | 0.245 x Scale(1.81) | weight branch x Scale(1.004e-4)           |
| <b>ttbar weight</b> | 0.245 x Scale(0.15) | weight branch x Scale(1.0)                |
| <b>DY Scale</b>     | 1.81 (empirical)    | 1.004e-4 (= 54900 x 6077.22 / 3.323e12)   |
| <b>ttbar Scale</b>  | 0.15 (empirical)    | 1.0 (already ~correct via 0.15 in Phase1) |

Muon  $p_T$

Figure 1: Muon  $p_T$

Muon eta

Figure 2: Muon eta

Muon mass

Figure 3: Muon mass

Tau pT

Figure 4: Tau pT

Tau eta

Figure 5: Tau eta

Tau mass  
Figure 6: Tau mass

# Single Distributions After Proton Merging

The following plots show individual kinematic distributions after requiring protons on both PPS arms.

## Variables shown

| Variable  | Description                   |
|-----------|-------------------------------|
| sist_mass | Invariant mass $m(\mu, \tau)$ |
| acop      | Acoplanarity                  |
| sist_pt   | System transverse momentum    |
| sist_rap  | System rapidity               |
| tau_pt    | Tau transverse momentum       |
| met_pt    | Missing transverse energy     |



# Invariant Mass (after proton requirements)

Mass after

Figure 7: Mass after

# Acoplanarity (after proton requirements)

Acop after

Figure 8: Acop after

# System pT (after proton requirements)

pT after

Figure 9: pT after

# System Rapidity (after proton requirements)

Rapidity after

Figure 10: Rapidity after

# Tau $p_T$ (after proton requirements)

Tau  $p_T$  after

Figure 11: Tau  $p_T$  after

# MET (after proton requirements)

MET after

Figure 12: MET after

## 33. BDT summary: workflow

### Pipeline

```
corrs_matrix.py --> TMVAClassification.C --> TMVAClassificationApplication.C
(feature QA)      (training)              (evaluation)
```

### Files

| File                            | Purpose                                                          |
|---------------------------------|------------------------------------------------------------------|
| corrs_matrix.py                 | Correlation heatmaps from variables2.txt to select features      |
| TMVAClassification.C            | TMVA factory: train BDT + Likelihood + Fisher + MLP + PDERS      |
| TMVAClassificationApplication.C | TMVA reader: apply trained weights, produce MVA score histograms |
| dataset/weights/*.weights.xml   | Serialized trained models                                        |

# BDT summary: training inputs and variables

## Input samples

| Role       | File                               | Tree |
|------------|------------------------------------|------|
| Signal     | MuTau_sinal_SM_2018_july.root      | tree |
| Background | background_total-protons_syst.root | tree |

## Event weights in TMVA

```
dataloader->SetSignalWeightExpression("weight*weights_bsm_sf[51]");  
dataloader->SetBackgroundWeightExpression("weight");  
// Global scale factors:  
double signalWeight = 0.00021;    double backgroundWeight = 104.;
```



## 9 training variables

| Variable                                                                       | Description                           |
|--------------------------------------------------------------------------------|---------------------------------------|
| sist_rap                                                                       | System rapidity                       |
| acop                                                                           | System acoplanarity                   |
| sist_pt                                                                        | System transverse momentum            |
| mu_pt, tau_pt                                                                  | Lepton transverse momenta             |
| sist_mass                                                                      | Invariant mass $m(\mu, \tau)$         |
| met_pt                                                                         | Missing transverse energy             |
| $\text{sist\_mass} - \sqrt{13000^2 * \text{xi\_arm1\_1} * \text{xi\_arm2\_1}}$ | Mass matching (central vs proton)     |
| $\text{sist\_rap} - 0.5 * \log(\text{xi\_arm1\_1} / \text{xi\_arm2\_1})$       | Rapidity matching (central vs proton) |

The last two compare the central  $\mu + \tau$  system with the forward proton kinematics. For true exclusive events they peak at zero.

# BDT results: using simple classifier

Only BDT with (AdaBoost) is enabled with the default hyperparameters

TMVAClassification → run\_all\_bdt.sh → bdt\_output.cpp

trains classifier, applies to each sample the results and produces corresponding histograms, plots the histogram of BDT results

# BDT results: using simple classifier

# BDT summary: configuration and output

## BDT (AdaBoost) hyperparameters

`NTrees=850, MaxDepth=3, MinNodeSize=2.5%, AdaBoostBeta=0.5,  
UseBaggedBoost, BaggedSampleFraction=0.5, GiniIndex, nCuts=20`

## Train/test split

- 6500 signal + 15000 background for training, rest for testing
- Random split, `NormMode=NumEvents`
- No additional preselection cuts inside TMVA

# TMVA methods used

5 classifiers are enabled in `TMVAClassification.C` by default:

| Method            | Type                  | Key idea                                                                                     |
|-------------------|-----------------------|----------------------------------------------------------------------------------------------|
| <b>BDT</b>        | Boosted Decision Tree | Ensemble of 850 shallow trees (depth 3), AdaBoost, GiniIndex splitting                       |
| <b>Likelihood</b> | Naive Bayes           | Per-variable PDF estimation (Spline2 interpolation), assumes uncorrelated inputs             |
| <b>Fisher</b>     | Linear discriminant   | Optimal linear combination of variables maximizing S/B separation                            |
| <b>MLP</b>        | Neural network        | Multilayer perceptron with tanh activation, hidden layer of N+5 neurons, 600 training cycles |
| <b>PDERS</b>      | Probability density   | Adaptive range search in multi-dim space, Gaussian kernel estimator                          |

All five produce per-event MVA scores stored in `TMVApp_sinal.root`.

## Application (TMVAClassificationApplication.C)

- Reads trained weights from dataset/weights/
- Computes derived matching variables in the event loop from xi\_arm1\_1, xi\_arm2\_1
- Per-event weight branch used as tree weight
- BDT score histogram: 20 bins in  $[-0.8, 0.8]$
- Output: TMVApp\_sinal.root (contains MVA\_BDT, MVA\_Likelihood, MVA\_Fisher, MVA\_MLP, MVA\_PDERS)

# BDT code files: TMVAClassification.C

TMVA factory script that trains the BDT classifier (and optionally other MVA methods). This is the core training step.

## Input

- Signal: `MuTau_sinal_SM_2018_july.root` (tree: `tree`)
- Background: `background_total-protons_syst.root` (tree: `tree`)
- ① Registers 9 training variables (kinematic + proton matching)
- ② Sets per-event weight expressions:
  - Signal: `weight * weights_bsm_sf[51]`, global scale = 0.00021
  - Background: `weight`, global scale = 104.0
- ③ Prepares train/test split: 6500 signal + 15000 background for training, random split
- ④ Books the BDT method with AdaBoost (NTrees=850, MaxDepth=3, GinIndex, nCuts=20)
- ⑤ Calls `TrainAllMethods()`, `TestAllMethods()`, `EvaluateAllMethods()`

## Output

- `TMVA_allBkg_Mutau_2018.root` - TMVA output with training/testing histograms, ROC curves
- `dataset/weights/TMVAClassification_BDT.weights.xml` - Serialized trained BDT model

# BDT code files: TMVAClassificationApplication.C

Applies the trained BDT model to the **signal** sample, producing MVA score histograms for evaluation.

## Input

- ROOT file: MuTau\_sinal\_SM\_2018\_july.root (tree: tree)
- Trained weights: dataset/weights/TMVAClassification\_BDT.weights.xml
- ① Creates a TMVA Reader and registers the same 9 variables used in training
- ② Reads branches: acop, sist\_pt, sist\_mass, sist\_rap, met\_pt, xi\_arm1\_1, xi\_arm2\_1, weight, tau\_pt, mu\_pt
- ③ For each event, computes the derived matching variables:
  - Mass matching:  $\text{sist\_mass} - \sqrt{13000^2 * \text{xi1} * \text{xi2}}$
  - Rapidity matching:  $\text{sist\_rap} - 0.5 * \log(\text{xi1}/\text{xi2})$
- ④ Skips events without valid protons on both arms ( $\text{xi} \leq 0$ )
- ⑤ Evaluates BDT score via `reader->EvaluateMVA("BDT method")`
- ⑥ Fills BDT score histogram (20 bins, [-0.8, 0.8])

## Output

- TMVApp sinal.root - Contains MVA BDT histogram for signal



# BDT code files: `run_bdt_all.C`

Standalone BDT application script that can be run on **any** sample type (signal, DY, ttbar, QCD, data). Unlike `TMVAClassificationApplication.C`, this handles the different branch naming and structure across sample types.

## Input

- Three arguments: `inputFile`, `outputFile`, `sampleType`
- Sample types: "signal", "dy", "ttbar", "qcd", "data"
- Trained weights: `dataset/weights/TMVAClassification_BDT.weights.xml`
- ① Creates a TMVA Reader with the same 9 training variables
- ② Adapts to sample-specific branch differences:
  - Data/QCD: muon pT branch = `muon_pt`, proton xi from vectors (`proton_multi_arm`, `proton_multi_xi`)
  - MC (signal/DY/ttbar): muon pT branch = `mu_pt`, proton xi from scalars (`xi_arm1_1`, `xi_arm2_1`)
- ③ Loops over events, requires protons on both arms (`xi1 > 0 && xi2 > 0`)
- ④ Computes derived matching variables, evaluates BDT score
- ⑤ Fills weighted histogram `MVA_BDT` (20 bins, `[-0.8, 0.8]`) using the weight branch

## Output

# BDT code files: run\_bdt\_all.sh

## Input

| Sample | Input file                                        |
|--------|---------------------------------------------------|
| Signal | MuTau_sinal_SM_2018_july.root (local)             |
| DY     | /eos/home-m/mblancco/samples_2018_mutau/DY_2018_U |
| ttbar  | /eos/home-m/mblancco/samples_2018_mutau/ttjets_20 |
| QCD    | /eos/home-m/mblancco/samples_2018_mutau/QCD_2018  |
| Data   | /eos/home-m/mblancco/samples_2018_mutau/Data_2018 |

## Output

| Output file        | Content           |
|--------------------|-------------------|
| TMVApp_sinal.root  | Signal BDT scores |
| TMVApp_DY.root     | DY BDT scores     |
| TMVApp_ttjets.root | ttbar BDT scores  |

# BDT code files: bdt\_output.cpp

Plotting script that creates a stacked histogram of BDT output scores, overlaying all backgrounds, signal, and data.

## Input

- TMVApp\_DY.root - DY BDT scores
- TMVApp\_QCD.root - QCD BDT scores
- TMVApp\_ttjets.root - ttbar BDT scores
- TMVApp\_sinal.root - Signal BDT scores
- TMVApp\_data.root - Data BDT scores

## What it does

- 1 Opens all five TMVApp\_\*.root files and extracts the MVA\_BDT histogram from each
- 2 Rebins all histograms by factor 4 (20 bins -> 5 bins)
- 3 Applies normalization: DY Scale( $1.004e-4$ ), Signal Scale(5000) for visibility
- 4 Builds a THStack with colored backgrounds: DY (yellow), QCD (red), ttbar (green)
- 5 Overlays signal (black, line width 3) and data (black points with errors)
- 6 Draws total background uncertainty band (gray hatched)

# Combine tool: workflow overview

## Full pipeline to compute the 95% CL upper limit

```
save_shapes.cpp      mutau_mass_shape.txt      combine
(build histograms)  --> (datacard)              --> (AsymptoticLimits)
      |                                         |
MuTau_shapes.root   higgsCombineMuTau.AsymptoticLimits.mH1
                    |
                    plot_single_limit.py
                    |
                    MuTau_limits.png / limits.json
```

## Commands run

*# 1. Build shape histograms*

```
g++ save_shapes.cpp -o save_shapes $(root-config --cflags --libs) && ./save_shapes
```

*# 2. Convert datacard to workspace*

```
text2workspace.py datacards/mutau_mass_shape.txt \
```

# Combine tool: save\_shapes.cpp

## Purpose

Produces the input ROOT file (`MuTau_shapes.root`) containing the per-sample invariant mass histograms that Combine reads as shapes.

## Inputs

| Sample | File                                                              |
|--------|-------------------------------------------------------------------|
| Data   | <code>Data_2018_UL_MuTau_nano_merged_proton_vars.root</code>      |
| DY     | <code>DY_2018_UL_MuTau_nano_merged_pileup_protons.root</code>     |
| ttbar  | <code>ttjets_2018_UL_MuTau_nano_merged_pileup_protons.root</code> |
| QCD    | <code>QCD_2018_UL_MuTau_nano_merged_proton_vars_new.root</code>   |
| Signal | <code>MuTau_sinal_SM_2018_july.root</code>                        |

## What it does

- 1 Loops over all events in each sample

# Combine tool: datacard (mutau\_mass\_shape.txt)

## Structure

```
imax 1    # 1 channel: mutau
jmax 3    # 3 backgrounds: DY, ttbar, QCD
kmax *    # all nuisances
```

## Shape sources

All histograms read from ../MuTau\_shapes.root:

| Process  | Histogram |
|----------|-----------|
| data_obs | m_data    |
| signal   | m_signal  |
| dy       | m_dy      |
| ttjets   | m_ttjets  |
| qcd      | m_qcd     |

# Combine tool: systematic uncertainties

## Lognormal (lnN) systematics

| Source        | Signal | DY    | ttbar | QCD |
|---------------|--------|-------|-------|-----|
| Luminosity    | 1.5%   | 1.5%  | 1.5%  | —   |
| tau ID vs jet | 5.0%   | 4.0%  | 4.0%  | —   |
| tau ID vs e   | 2.5%   | 2.5%  | 2.5%  | —   |
| tau ID vs mu  | 3.1%   | 3.2%  | 2.6%  | —   |
| Muon trigger  | 0.5%   | 0.07% | 0.07% | —   |
| Muon ID+Iso   | 0.02%  | 0.04% | 0.04% | —   |
| Muon reco     | 0.06%  | 0.05% | 0.05% | —   |
| Proton xi     | 50%    | 50%   | 50%   | 50% |

## Normalization floats (rateParam)

- `beta_dy`, `beta_tt`, `beta_qcd` — free floating in  $[0, 5]$ , constrained by `dy_norm` (30%), `ttjets_norm` (50%), `qcd_norm` (50%)

## Combine tool: results

Method: AsymptoticLimits at 95% CL,  $m_H = 120$  GeV

| Estimate                 | Limit on signal strength $r$ |
|--------------------------|------------------------------|
| Expected $-2$            | 3156                         |
| Expected $-1$            | 3987                         |
| <b>Expected (median)</b> | <b>4000</b>                  |
| Expected $+1$            | 8050                         |
| Expected $+2$            | 12131                        |
| <b>Observed</b>          | <b>13891</b>                 |

Signal strength  $r$  is defined as  $\sigma_{\text{obs}}/\sigma_{\text{SM}}$ . The large values indicate the analysis is not yet sensitive to the SM signal with current statistics and selection.



MuTau limits

Figure 13: MuTau limits

# Combine tool: `plot_single_limit.py`

## Purpose

Reads the Combine output ROOT file and produces a summary text file and a ROOT-based limit plot.

## Input

- `higgsCombineMuTau.AsymptoticLimits.mH120.root` — Combine output tree limit
- `--mass 120` — x-axis mass value
- `--output MuTau_limits` — output file prefix

## What it does

- 1 Iterates over entries in the `limit` tree, identifies each quantile by `quantileExpected`:
  - $-1 \rightarrow$  observed,  $0.5 \rightarrow$  median expected,  $0.16/0.84 \rightarrow \pm 1$ ,  $0.025/0.975 \rightarrow \pm 2$
- 2 Writes a plain-text summary table (`MuTau_limits.txt`)
- 3 Draws a `TMultiGraph` with yellow ( $\pm 2$ ) and green ( $\pm 1$ ) bands, dashed expected line, and observed point
- 4 Saves as PNG and PDF